Deep Learning for MNIST Digit Classification

Project Report

Author: Danish Touseeq

A PyTorch-based study of linear classification, multilayer perceptrons, and convolutional neural networks for handwritten digit recognition.

Abstract

This project explores handwritten-digit classification on the MNIST dataset using PyTorch. It compares a simple linear classifier, a multilayer perceptron (MLP), and a convolutional neural network (CNN). The notebook includes data loading, model construction, training loops, validation, test evaluation, and experiments with selected MLP hyperparameters.

1. Introduction

Handwritten digit recognition is a common introductory computer-vision task. MNIST contains grayscale images of digits from 0 through 9. The goal is to learn a model that maps each image to its correct digit label.

2. Dataset and preprocessing

The project uses the MNIST dataset through torchvision.datasets.MNIST. Images are converted to tensors and have a spatial size of 28 × 28 pixels with one grayscale channel. A flattened image therefore has 784 features. The task has ten output classes (digits 0–9). The dataset loader downloads the data when it is not already present locally.

3. Models

Linear classifier

The linear baseline maps the 784 input pixel values directly to ten output scores. It provides a simple reference model but cannot learn hierarchical spatial features of a deeper network.

Multilayer perceptron (MLP)

The MLP uses fully connected layers with ReLU activations. The notebook experiments with different hidden-layer sizes, learning rates, and batch sizes. Cross-entropy loss is used for classification.

Convolutional neural network (CNN)

The CNN uses two convolutional layers with ReLU activations and max-pooling, followed by fully connected layers. Convolutional layers can learn local image patterns, while pooling reduces spatial dimensions before classification.

4. Training and evaluation

The notebook uses PyTorch data loaders to process examples in mini-batches. Training follows the standard optimization loop: forward pass, loss calculation, backpropagation, and parameter update. Evaluation is performed without gradient calculation. Accuracy is computed as the fraction of predictions whose predicted class matches the target label. For a fair comparison, use the same train/test split and evaluation procedure for each model, record hyperparameters, and select MLP settings using validation performance rather than test-set results.

5. Results

The provided notebook did not include verified final metrics that could be reported here. No accuracy values are claimed in this report. After running the notebook, add measured test accuracy and training time for each model.

Model	Input	Main components	Test accuracy
Linear classifier	784 flattened features	Single linear layer	Record after run

Model	Input	Main components	Test accuracy
MLP	784 flattened features	Fully connected + ReLU	Record after run
CNN	1 × 28 × 28 image	Convolution + pooling + dense layers	Record after run

6. Discussion

The linear classifier is a useful baseline. The MLP can model nonlinear relationships between pixels, while the CNN preserves the two-dimensional structure of images and learns local visual features. The expected comparison should be based on measured results rather than assumptions. Training time, random seeds, hardware, and hyperparameters should be documented to make results reproducible.

7. Limitations and future work

• The original notebook includes legacy code that expects a local mnist.pkl.gz file; the organized version uses torchvision to download MNIST instead.

• Some experiment loops train many configurations and may take considerable time.

• Results should be recorded from a fresh, complete run.

• Future work could include learning-rate scheduling, Adam optimization, normalization, confusion matrices, per-class precision/recall, and saving the best model checkpoint.

8. Conclusion

This project provides a practical comparison framework for linear, fully connected, and convolutional neural networks on handwritten-digit classification. It demonstrates dataset loading, model definition, optimization, and evaluation in PyTorch. Final performance conclusions should be added after all models have been executed and their metrics recorded.

Reproducibility

Install the dependencies listed in requirements.txt, then run the notebook in notebooks/ or use the standalone training script in src/. The first run may download MNIST and requires an internet connection. Exact results can vary with random seeds, software versions, and hardware.